ASP.NET Core Assignment

Question 1: What is the difference between a Multiple Page Application (MPA) and a Single Page Application (SPA)? For the three projects recommended by your team, identify which approach is better for each project and explain why.

Answer:

A **Multiple Page Application (MPA)** is a web application in which navigation between pages usually sends a request to the server, and the server returns a new HTML page. Each major page is loaded separately. MPAs are often straightforward to build, work well with server-side rendering, and can be suitable for content-heavy websites that need search-engine visibility.

A **Single Page Application (SPA)** loads the main application page once and then updates its content dynamically using JavaScript, usually by communicating with a backend API. SPAs can provide smooth interactions and a responsive user experience, but they may require more frontend complexity and careful handling of routing, loading states, and search-engine optimization.

Recommended approach for the three projects

Project	Recommended approach	Reason
1. Business Management System	**MPA** (ASP.NET Core MVC)	This system mainly contains forms, tables, reports, and administrative pages for managing customers, products, orders, users, and sales. An MPA is a practical choice because server-rendered pages make these workflows straightforward and keep the application relatively simple to develop and maintain.
2. E-Commerce API & Admin Dashboard	**SPA** for the dashboard; a hybrid or server-rendered approach for public storefront pages	The admin dashboard benefits from dynamic updates when managing products, orders, search, and analytics. The public storefront also needs discoverable product pages, so server-side rendering or a hybrid approach can improve SEO while keeping interactive features such as the cart responsive.
3. Appointment Booking System	**SPA**	Users frequently select services, dates, and available time slots. A SPA can update available slots and booking details without reloading the entire page, creating a smoother and more interactive booking experience. The backend must still validate availability before confirming a booking.

Conclusion: The best choice depends on the application's users and features. MPA is suitable for structured, page-based management workflows, while SPA is useful when users need frequent dynamic interactions. A hybrid approach can combine the strengths of both when a project has different needs in its public and administrative areas.

Question 2: What is the difference between REST and gRPC?

Answer:

REST (Representational State Transfer) is an architectural style commonly used to build web APIs. REST APIs usually communicate over HTTP and exchange data in JSON format. They use standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE. REST is widely supported and is convenient for communication between web or mobile clients and backend services.

gRPC (Google Remote Procedure Call) is a high-performance remote procedure call framework. It commonly uses HTTP/2 and Protocol Buffers (Protobuf) to define services and serialize messages. It supports features such as strongly typed contracts, streaming, and efficient communication between services.

Key differences

REST	gRPC
Usually exchanges JSON (though other formats are possible).	Usually uses Protocol Buffers for compact, structured messages.
Commonly uses HTTP methods and resource-based URLs.	Uses defined service methods and procedure calls.
Broad browser, mobile, and tooling support; easy to test with common HTTP tools.	Efficient and well suited to service-to-service communication; browser use often needs gRPC-Web or a proxy.
A strong choice for public APIs and frontend-to-backend communication.	A strong choice for internal microservices that need efficient communication or streaming.

When should we use each one?
Use REST when building APIs for browsers, mobile applications, or external developers, especially when simplicity and broad compatibility are important. Use gRPC when backend services need fast, efficient communication, strongly defined contracts, or streaming. The choice depends on the system's requirements; neither option is always better.

Conclusion: REST is generally simpler and more widely accessible for web APIs, while gRPC offers efficient communication and strong service contracts, making it useful for communication between backend services.